Proje: Task Manager API

Eğitim boyunca tek bir proje üzerinde çalışacağız: Task Manager. Görev yönetim sistemi. Seçmemin sebebi basit — gerçek hayat projelerinde her zaman olan CRUD işlemlerini, katmanlı mimariyi ve pattern kullanımını en sade haliyle gösteriyor. Karmaşık bir domain bilgisi gerektirmiyor, ama backend'in tüm temel kavramlarını barındırıyor.

Kullanıcılar görev oluşturabilecek, listeleyebilecek, tamamlayabilecek ve silebilecek. Her görevin bir başlığı, durumu ve oluşturulma tarihi olacak. Üç oturum sonunda ortada çalışan, test edilebilir, eksiksiz bir REST API duruyor olacak.

SESSION 1 — Veri Nedir, Nerede Durur?

Entity ve Repository | Süre: 1.5 Saat

Bugün size bir soru sormak istiyorum: Bir kullanıcı uygulamaya "görev ekle" dediğinde, o veri nereye gidiyor?

Cevap basit gibi görünüyor — veritabanına. Ama bu yolculuk nasıl gerçekleşiyor? Java'daki bir nesne, disk üzerindeki bir satıra nasıl dönüşüyor? Bu sorunun cevabını bugün birlikte bulacağız.

Büyük Resim: Katmanlı Mimari

Kod yazmaya başlamadan önce şunu kafanıza kazımanızı istiyorum:

Kullanıcı isteği önce Controller'a gelir. Controller kapıdaki görevlidir; gelen isteği karşılar, ilgili yere yönlendirir. Oradan Service'e geçer. Service iş yapan yerdir; karar burada verilir, kurallar burada uygulanır. Service'den Repository'ye iner. Repository veritabanıyla konuşan tek yerdir, başkası bu işe karışmaz. Repository de veritabanına ulaşır.

Bu şemayı şimdi bir yere yazın. Eğitim boyunca sürekli döneceğiz.

Neden bu katmanlar var? Çünkü her şeyi tek bir yere yazdığınızda, bir şeyi değiştirmek istediğinizde her şeyi kırma riskiyle karşı karşıya kalırsınız. Katmanlar, değişikliği izole etmek için var. Veritabanını değiştirmek mi istiyorsunuz? Sadece

Repository'ye dokunursunuz. İş kuralı değişti mi? Sadece Service'e bakarsınız. Bu ayrım sizi gece uyandıran üretim hatalarından korur.

Entity: Verinin Java'daki Yüzü

Entity'yi şöyle düşünün: veritabanında bir tablo var, o tablonun her satırı bir Java nesnesiyle eşleşiyor. O Java nesnesi bizim Entity'miz.

```
@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Enumerated(EnumType.STRING)
    private TaskStatus status;

    private LocalDateTime createdAt;

    // getter ve setter'lar
}
```

```
public enum TaskStatus {
    PENDING,
    IN_PROGRESS,
    DONE
}
```

Anotasyonların ne anlama geldiğini birlikte okuyalım.

@Entity, Spring'e "bu sınıf bir veritabanı tablosuna karşılık gelir" diyor. @Table ile tablonun adını belirliyoruz; yazmadan da olur ama açık seçik olmak iyidir. @Id ile

"bu alan primary key" diyoruz. @GeneratedValue ile ID'yi sen üretme, veritabanı otomatik arttırsın diyoruz. @Enumerated(EnumType.STRING) ile enum değerini veritabanına sayı olarak değil, "PENDING" veya "DONE" şeklinde metin olarak kaydet diyoruz. Sayı olarak saklasaydık, ne anlama geldiğini anlamak için her seferinde koda bakmak zorunda kalırdınız.

Şimdi durun. Bu sınıfı yazarak veritabanına kayıt yapabilir miyiz? Hayır. Entity sadece veriyi taşır. Kaydetme, okuma, silme işlemleri için başka bir şeye ihtiyacımız var.

Repository: Veritabanıyla Tek Konuşan

Repository katmanı, veritabanı ile kurduğumuz tek iletişim noktası. Başka hiçbir katman doğrudan veritabanına gitmez. Bu kural çiğnenmez.

```
@Repository
public interface TaskRepository extends JpaRepository<Task, L
ong> {

    List<Task> findByStatus(TaskStatus status);

    Optional<Task> findByTitle(String title);
}
```

Şimdi büyük ihtimalle aklınızda şu soru oluştu: neden interface? Bu interface'i kim implement ediyor?

Harika bir soru. Cevap: Spring bunu sizin yerinize yapıyor. JpaRepository'yi extend ettiğinizde Spring, uygulama ayağa kalkarken bu interface için otomatik bir implementasyon üretiyor. findAll(), save(), deleteById(), findById() gibi onlarca metod zaten içinde hazır geliyor. Sizin findByStatus yazdığınızda ise Spring metodun ismini okuyarak "status alanına göre filtrele" demek istiyorsun anlıyor ve SQL sorgusunu kendisi yazıyor.

Optional ne anlama gelir? "Bu sorgu sonuç döndürebilir de döndürmeyebilir de" demek. Bunu görmezden gelmek NullPointerException'a davetiye çıkarmaktır. Sonucu her zaman kontrol edin.

Pratik

Task entity'sini ve TaskRepository'yi oluşturun. application.properties dosyasına şu ayarları ekleyin:

spring.datasource.url=jdbc:h2:mem:taskdb spring.datasource.driver-class-name=org.h2.Driver spring.h2.console.enabled=true spring.jpa.hibernate.ddl-auto=create-drop spring.jpa.show-sql=true

Uygulamayı başlatın ve http://localhost:8080/h2-console adresine gidin. tasks tablosunun otomatik oluştuğunu görün. show-sql=true ayarını özellikle fark edin — Spring'in arka planda ne SQL çalıştırdığını görmenizi sağlar. Başlangıçta bu ayarı açık tutmanızı öneririm.

SESSION 2 — Kimin İşi Ne?

Service Katmanı ve Strategy Pattern | Süre: 1.5 Saat

Geçen oturumda Entity ve Repository'yi gördük. Bugün şu soruyu yanıtlayacağız: iş mantığı nereye yazılır?

"İş mantığı" soyut bir kavram gibi duyuluyor. Somutlaştıralım: "Bir görev tamamlandığında, oluşturulma tarihinden 7 günden fazla geçmişse yöneticiye bildirim gönder." İşte bu bir iş kuralı. Bu kural kodun hangi katmanına girer?

Controller'a mı? Hayır, Controller'ın işi HTTP isteğini karşılamak. Repository'ye mi? Kesinlikle hayır, Repository sadece veritabanıyla konuşur. Cevap: Service.

Service Katmanı

```
@Service
@RequiredArgsConstructor
public class TaskService {

    private final TaskRepository taskRepository;

    public Task createTask(String title) {
        Task task = new Task();
        task.setTitle(title);
        task.setStatus(TaskStatus.PENDING);
        task.setCreatedAt(LocalDateTime.now());
```

```
        return taskRepository.save(task);
    }

    public Task getTaskById(Long id) {
        return taskRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Task bul
unamadı: " + id));
    }

    public Task completeTask(Long id) {
        Task task = getTaskById(id);
        task.setStatus(TaskStatus.DONE);
        return taskRepository.save(task);
    }

    public List<Task> getAllTasks() {
        return taskRepository.findAll();
    }
}
```

@RequiredArgsConstructor şu an için şunu yapıyor: final olarak tanımlanan alanlar için otomatik constructor üretiyor. Spring bu constructor'ı kullanarak TaskRepository'yi inject ediyor. Buna Constructor Injection deniyor ve en temiz dependency injection yöntemi bu.

orElseThrow satırına dikkat edin. Olmayan bir görevi istediğinizde boş bir Optional dönüyor. Biz bunu susturmak yerine açıkça bir hata fırlatıyoruz. İleride bunu daha da geliştireceğiz.

Strategy Pattern: Değişen Davranışı Yönetmek

Şimdi size gerçek bir problem tanıtmak istiyorum. Vakti geldiğinde hepinizin karşılaşacağı bir durum.

Senaryo: Görevleri listelemek istiyoruz. Ama bazı kullanıcılar tarihe göre, bazıları başlığa göre, bazıları duruma göre sıralanmış liste istiyor.

İlk akla gelen çözüm şöyle görünür:

```
// Bunu yazmayın
public List<Task> getTasks(String sortType) {
    List<Task> tasks = taskRepository.findAll();

    if (sortType.equals("date")) {
        tasks.sort(Comparator.comparing(Task::getCreatedAt));
    } else if (sortType.equals("title")) {
        tasks.sort(Comparator.comparing(Task::getTitle));
    } else if (sortType.equals("status")) {
        // ...
    }
    // Yarın "önceliğe göre" isteği gelirse?
    // Bu metoda tekrar gireceksiniz.
    // Her yeni özellik mevcut kodu değiştirmek demek.
    // Bu tehlikelidir.

    return tasks;
}
```

Bu kod ilk bakışta zararsız görünüyor. Ama 3 ay sonra 8 farklı sıralama seçeneği eklendiğinde, üstüne bir de if içinde bug çıktığında, bu metodu kim düzeltmeye cesaret edebilir?

Strategy Pattern tam bu problem için var. Fikir şu: değişen davranışı ayrı sınıflara taşı. Her sıralama yöntemi kendi sınıfında yaşasın. Yeni bir sıralama eklemek istediğinde mevcut koda dokunmadan yeni bir sınıf yazmak yeterli olsun.

Adım 1 — Ortak arayüzü tanımla:

```
public interface TaskSortStrategy {
    List<Task> sort(List<Task> tasks);
}
```

Bu interface bir sözleşme. "Sıralama yapan her şey bu metodu implemente etmek zorunda" diyor.

Adım 2 — Stratejileri yaz:

```
@Component("byDate")
public class SortByDate implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
            .sorted(Comparator.comparing(Task::getCreatedAt))
            .toList();
    }
}
```

```
@Component("byTitle")
public class SortByTitle implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
            .sorted(Comparator.comparing(Task::getTitle))
            .toList();
    }
}
```

Adım 3 — Service'e entegre et:

```
@Service
@RequiredArgsConstructor
public class TaskService {

    private final TaskRepository taskRepository;
    private final Map<String, TaskSortStrategy> sortStrategie
s;
    // Spring, @Component isimlerini ("byDate", "byTitle")
    // otomatik olarak Map'in key'i yapıyor.

    public List<Task> getTasks(String sortType) {
        List<Task> tasks = taskRepository.findAll();
```

```
        TaskSortStrategy strategy = sortStrategies
            .getOrDefault(sortType, sortStrategies.get("byDat
e"));

        return strategy.sort(tasks);
    }
}
```

Şimdi şunu fark edin: yarın "önceliğe göre sırala" özelliği istense ne olur? TaskService'e tek satır dokunmadan sadece yeni bir SortByPriority sınıfı yazarsınız. Var olan kod bozulma riskiyle karşılaşmaz.

Yazılımda buna Open/Closed Principle deniyor: kod genişlemeye açık, değişime kapalı olmalı. Strategy Pattern bunu mümkün kılmanın en temiz yollarından biri.

Pratik

SortByStatus adında üçüncü bir strateji yazın. Sıralama mantığı şöyle olsun: önce PENDING, sonra IN_PROGRESS, en son DONE gelsin. Bunu doğru yazdığınızda TaskService'e tek satır eklemeden yeni strateji çalışıyor olacak. Eğer Service'i değiştirmeniz gerekiyorsa bir şeyi yanlış yapıyorsunuz demektir.

SESSION 3 — Dışarıya Açılıyoruz

Controller ve Uçtan Uca Akış | Süre: 1.5 Saat

Son iki oturumda arka planı inşa ettik. Veriyi temsil etmeyi, saklamayı, iş mantığını uygulamayı öğrendik. Ama hâlâ dışarıdan erişilemeyen bir uygulamamız var. Bugün HTTP kapısını açacağız.

HTTP'yi Hızlıca Anlayalım

REST API yazmak için HTTP'nin temellerini bilmek şart.

GET veri okur, hiçbir şeyi değiştirmez. POST yeni bir şey oluşturur. PUT var olanı tamamen günceller. PATCH var olanın bir kısmını günceller. DELETE siler.

URL yapısı bir kural değil, bir anlaşma. Biz şu kuralı benimsiyoruz:

GET /api/tasks tüm görevleri listeler. GET /api/tasks/5 id'si 5 olan görevi getirir. POST /api/tasks yeni görev oluşturur. PATCH /api/tasks/5/complete görevi tamamlar. DELETE /api/tasks/5 görevi siler.

HTTP durum kodları da önemli. Uygulamanız her durumda 200 dönüyorsa bir şeyler yanlış demektir. 200 her şey yolunda demek, 201 bir şey oluşturuldu demek, 400 isteğin hatalı demek, 404 aradığın şey yok demek, 500 bizim tarafımızda bir şeyler patladı demek.

Controller

```
@RestController
@RequestMapping("/api/tasks")
@RequiredArgsConstructor
public class TaskController {

    private final TaskService taskService;

    @GetMapping
    public List<Task> getAllTasks(
            @RequestParam(defaultValue = "byDate") String sor
t) {
        return taskService.getTasks(sort);
    }

    @GetMapping("/{id}")
    public Task getTask(@PathVariable Long id) {
        return taskService.getTaskById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Task createTask(@RequestBody TaskCreateRequest req
uest) {
        return taskService.createTask(request.title());
    }
```

```
    @PatchMapping("/{id}/complete")
    public Task completeTask(@PathVariable Long id) {
        return taskService.completeTask(id);
    }
}

record TaskCreateRequest(String title) {}
```

Controller'a bakın: tek satır iş mantığı yok. Her metod şunu yapıyor — isteği al, service'e ilet, cevabı dön. Bu kasıtlı bir tercih. Controller'da iş mantığı olmaz. Controller sadece trafik polisi.

@RequestParam URL'deki ?sort=byTitle gibi parametreleri okur. @PathVariable URL'deki /tasks/5 gibi dinamik parçaları okur. @RequestBody HTTP isteğinin gövdesindeki JSON'u Java nesnesine çevirir.

Uçtan Uca Akışı Gözlemleyelim

Bir POST /api/tasks isteği geldiğinde ne olur?

HTTP isteği gelir: POST /api/tasks, gövdede {"title": "Rapor yaz"}. TaskController.createTask() devreye girer, JSON gövdesi TaskCreateRequest nesnesine dönüşür. taskService.createTask("Rapor yaz") çağrılır, Service yeni bir Task nesnesi oluşturur, status'u PENDING ve zamanı şimdiki zaman olarak set eder. taskRepository.save(task) çağrılır, JPA INSERT SQL'ini üretir ve veritabanına yazar. Kaydedilen Task nesnesi geri döner, Jackson kütüphanesi bunu JSON'a çevirir ve HTTP 201 status koduyla response gönderilir.

Her katman sadece kendi işini yaptı. Hiçbiri diğerinin nasıl çalıştığını bilmek zorunda kalmadı. İşte bu ayrışma, kodun büyüdükçe yönetilebilir kalmasını sağlıyor.

Hata Yönetimi

Şu an uygulamamızda olmayan bir ID istenirse Spring 500 hatası döndürüyor. Bu kabul edilemez. Kullanıcıya "bizim tarafımızda bir şeyler patladı" demek yerine "böyle bir kayıt yok" demek zorundayız.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleNotFound(RuntimeExceptio
n ex) {
        return Map.of("error", ex.getMessage());
    }
}
```

@RestControllerAdvice uygulamadaki tüm controller'lardan fırlayan hataları merkezi olarak yakalar. Her controller'a ayrı ayrı try-catch yazmak zorunda kalmazsınız. Hata yönetimini merkezi tutmak iyi bir alışkanlık — yeni bir hata türü eklemek istediğinizde tek bir yere gelirsiniz.

Canlı Demo

Şimdi Postman veya tarayıcı üzerinden birlikte test edelim.

Birkaç görev oluşturun, farklı başlıklar verin. Tümünü listeleyin. sort=byTitle parametresiyle başlığa göre sıralanmış listeyi görün. sort=byDate ile tarihe göre sıralanmış listeyi görün. Olmayan bir ID'ye GET atın, 404 gelsin. Bir görevi tamamlayın.

Strategy Pattern'in farkını burada somut olarak görün: URL'deki tek parametre değişikliğiyle tamamen farklı sıralama mantığı devreye giriyor. Ve siz TaskService koduna tek satır dokunmadınız.

Öğrendiklerimizin Özeti

Entity, verinin Java'daki temsilidir. Veritabanı tablosuyla bire bir eşleşir. İş mantığı taşımaz.

Repository, veritabanıyla konuşan tek katmandır. Spring Data JPA sayesinde SQL yazmadan hazır metodlara sahipsiniz. Ama arka planda SQL üretildiğini unutmayın.

Service, iş mantığının evidir. Karar burada verilir, kurallar burada uygulanır. Controller ile Repository arasındaki koordinatör odur.

Strategy Pattern, değişen davranışları ayrı sınıflara taşıyarak mevcut kodu koruma altına almanın yoludur. Yeni özellik eklemek, var olanı bozmak anlamına gelmemelidir.

Controller, HTTP katmanıdır. İş mantığı içermez. Sadece isteği alır, service'e iletir, cevabı döndürür.

Bundan Sonra Ne Öğrenmeli?

Bu eğitim bir başlangıç noktası. Burada öğrendikleriniz gerçek projelerde karşılaşacağınız yapıların temelini oluşturuyor.

Kısa vadede şunlara bakın: DTO pattern ve neden entity'leri direkt döndürmemek gerektiği, Lombok ile boilerplate koddan kurtulmak, Bean Validation ile input kontrolü.

Orta vadede: Spring Security ile kimlik doğrulama, JUnit ve Mockito ile test yazmak, pagination ve filtreleme.

Uzun vadede: Docker ile uygulama paketleme, CI/CD pipeline, microservice mimarisi.